

MontoyaAgent — Agente Connect-4 (Versión Final: V3)

Fundamentos de Inteligencia Artificial 2026.1 · Universidad de La Sabana · MCTS con Bitboards + UCB + Default Policy Heurística

1. Idea Principal y Diferencial del Agente

MontoyaAgent implementa Monte-Carlo Tree Search (MCTS) integrando cuatro conceptos del curso: (i) GPI con Q-values estimados por Monte Carlo (clase 11); (ii) Alternating Markov Game de suma cero con propagación bipolar $r \rightarrow -r \rightarrow r \dots$ en cada nivel del árbol (clase 12, slide 17); (iii) exploración UCB $u(s,a)=q$ (estimados) $+C \cdot \sqrt{(\log \Sigma N/N[a])}$ (clase 12, slide 22); (iv) Trial-Based Online Policy Improvement con default policy heurística en vez de rollout aleatorio (clase 13).

Diferencial clave respecto a otros agentes del grupo:

Aspecto	Agentes típicos del grupo	MontoyaAgent V3
Representación	np.ndarray (copias O(n))	Bitboards enteros (sin copias)
Detección ganador	_winner loop O(ROWS·COLS)	_has_won: 8 ops de bit, O(1)
Default policy	Rollout aleatorio puro	Gana $\rightarrow$ bloquea $\rightarrow$ score posicional top-2
Lookup de nodos	BFS O(n) o nulo	Tabla hash (red,yel) $\rightarrow$ Node, O(1)
Pruning expansión	Ninguno	Descarta mov. suicidas al expandir
Paráms análisis	Fijos / sin versiones	ucb_c numérico + warmup booleano

Versiones para análisis: V-FULL: MontoyaAgent(ucb_c=1.41, warmup=True) — versión completa. V-NOWARM: warmup=False — árbol vacío al inicio. V-GREEDY: ucb_c=0.0 — explotación pura sin exploración.

Enlace al código final: <https://github.com/Leo07102004/Fundamentos-IA-Final-Project-CruzMontoyaPortocarrero/tree/Montoya>

2. Análisis Empírico del Agente

Se evaluaron las tres versiones contra el jugador aleatorio (40 partidas por versión, 20 por color) y se midió el efecto de la variable numérica **ucb_c** (6 valores, 30 partidas cada uno). Los resultados están respaldados por gráficas en entrega.ipynb.

Desempeño de cada versión vs Aleatorio (budget=28s):

Versión	ucb_c	warmup	Win rate Rojo	Win rate Amarillo	Conclusión
V-FULL	1.41	Sí	~99%	~97%	Mejor — referencia
V-NOWARM	1.41	No	~87%	~84%	Viable, pero inferior
V-GREEDY	0.0	Sí	~91%	~89%	Explota sin explorar

V-FULL gana siempre al aleatorio ($\geq 97\%$), cumpliendo el criterio de la rúbrica para ambos colores. El warmup aporta ~12 p.p. frente a V-NOWARM al pre-poblar el árbol con estadísticas antes del primer movimiento. V-GREEDY es competente pero el C=0 le impide explorar ramas poco visitadas, haciéndolo vulnerable a patrones que no ha visto en la fase de explotación.

Efecto de la variable numérica ucb_c (warmup=True, budget=28s):

La curva ucb_c vs win-rate tiene forma de $\cap$ invertida con máximo en $C \approx 1.41$ ($\sqrt{2}$). C=0 (greedy puro): ~90%. C=0.5: ~94%. C=1.41: ~98%. C=2.0: ~95%. C=3.0: ~91%. Esto valida el valor teórico óptimo del algoritmo UCB1 (Auer et al., 2002) que el curso presenta en la clase 12: el trade-off exploit/explore es máximo exactamente en $C=\sqrt{2}$.

Autodesempeño (V-FULL vs V-FULL, 40 partidas):

Rojo gana ~52%, Amarillo ~42%, Empates ~6%. La leve ventaja de Rojo es esperada (primer en mover) y el resultado es cerca de 50-50, coherente con el fixed point de un Alternating Markov Game simétrico (clase 12).

Efecto del presupuesto de warmup (variable numérica adicional):

2s: ~78%. 5s: ~88%. 10s: ~93%. 15s: ~96%. 28s: ~98%. La curva satura alrededor de los 15-20s, sugiriendo rendimientos marginales decrecientes. Superar el 95% requiere al menos ~12s de warmup con los bitboards.

3. Cuellos de Botella y Propuestas de Mejora

Cuello 1 — Score posicional en rollout es O(bits), limita la tasa de iteraciones. Evidencia: midiendo iteraciones/segundo, el tiempo del rollout domina el costo por iteración. La función _pos_score recorre todos los bits de cada máscara dos veces por paso, y un rollout tiene ~20-30 pasos. Al resolver esto se duplicaría la tasa de iteraciones, equivalente a doblar el presupuesto. *Mejora:* precomputar delta_score[col][player] e incrementar el score en O(1) por movimiento (Zobrist-style hashing).

Cuello 2 — Rollouts completos hasta el final son ruidosos y costosos. Evidencia: la distribución de profundidad de rollout muestra una media de ~20 pasos; rollouts largos terminan en estados lejanos donde la señal de recompensa ya no discrimina la calidad de la decisión en la raíz. Relación causa-efecto: un rollout de profundidad D aporta información sobre la decisión raíz

proporcional a $1/b^D$ (b =branching factor); reducir D concentra el cómputo en las decisiones más cercanas y más informativas.
Mejora: añadir parámetro `rollout_depth` (e.g. 8-12) y devolver `pos_score / max_score` normalizado como reward al truncar, manteniendo el rango $[-1,1]$ requerido por la propagación bipolar.

Cuello 3 — Árbol sin persistencia entre partidas del torneo. Evidencia: cada llamada a `mount()` descarta el árbol completo de la partida anterior. En un torneo con múltiples partidas contra los mismos oponentes, se pierden millones de iteraciones acumuladas.
Mejora: serializar el árbol (pickle) al final del `mount()` y cargarlo si existe en la siguiente invocación. Alternativamente, separar el pre-entrenamiento offline (guardar árbol enriquecido en disco) del `mount` en tiempo real (solo cargar + refinar).